

CryptoCore-16

16-bit Cryptographic Coprocessor Design in VHDL

Course: Hardware Design and Digital Systems
Under supervision of professor **Howaida Abdul Latif**

Submitted by:

Name	Student ID	Email	Work
Hatem Ayman Nabil	20812023100716	hatem.ayman.508@gmail.com	Project coordination and testing, combinational logic block
Amal Ragab El-Muhammady	20812023200455	ragabaml242@gmail.com	Register, crypto_core16 TB
Alsayed Abdul Raouf Ahmed	20812023100463	elsayed2005920@gmail.com	Nonlinear lookup table, Crypto_core16
Alaa Muhammad Thabet	20812023200555	am5897667@gmail.com	Shifter unit / shifter TB
Eyad Islam El-Taher	20812023100040	eltahereyad0@gmail.com	ALU & ALU TB & Report

Table of Contents

1	Assignment Aim	3
2	The Problem and Solution	4
2.1	Problem Description	4
2.1.1	The Need for a Cryptographic Coprocessor	4
2.1.2	Challenges in Hardware-Based Cryptography	4
2.2	Proposed Solution	5
2.2.1	System Architecture and Approach	5
3	Implementation	7
3.1	System Architecture	7
3.1.1	Input Register (Pipeline Stage)	7
3.1.2	Register File (16x16)	8
3.1.3	Arithmetic Logic Unit (ALU)	13
3.1.4	Shifter Unit	16
3.1.5	Non-Linear Lookup Unit (LUT / S-Box)	20
3.2	Design Details	24
3.2.1	Component Interaction and Data Flow	24
3.2.2	Control Signals and Opcode Usage	25
3.2.3	Instruction Format	26
3.3	VHDL Code Description	26
3.3.1	Overall Code Structure	26
3.3.2	Full Signal Flow: From Input to Output	26
4	Inputs and Outputs	28
4.1	Primary Interface Signals	28
4.2	Internal Signal Summary	29
5	Schematic Diagram	30
5.1	Quartus® Prime Schematic Overview	30
5.2	Combinational Logic Block Schematic	31

6	Final Simulation Results	32
6.1	Simulation Environment	32
6.2	Combinational Logic Block Simulation	32
6.2.1	Combinational logic block TB implementation	32
6.3	Top-Level System Simulation	35
6.3.1	Top-Level Source Code: <code>crypto_core16.vhd</code>	35
6.3.2	Testbench Code: <code>crypto_core16_tb.vhd</code>	37
6.3.3	Simulation Waveform	39
7	Conclusion	41

1. Assignment Aim

The primary objective of this project is to design, implement, and verify a **16-bit Cryptographic Coprocessor** (CryptoCore-16) using VHDL (VHSIC Hardware Description Language). The coprocessor is intended to accelerate computationally intensive cryptographic operations that would otherwise be a bottleneck for a general-purpose central processing unit (CPU).

Specifically, the project aims to:

1. Design a complete hardware architecture capable of executing a defined set of 12 cryptographic and arithmetic instructions.
2. Implement a modular VHDL design consisting of five major components: an Input Pipeline Register, a 16×16 Register File, an Arithmetic Logic Unit (ALU), a Shifter Unit, and a Non-Linear Lookup Table (LUT / S-Box).
3. Integrate all components into a single top-level entity (`crypto_core16`) that orchestrates data flow and control.
4. Validate the correctness of each component independently through dedicated VHDL testbenches, and verify the full system behaviour through integration simulation.
5. Demonstrate how hardware-level parallelism and pipelining improve the throughput and latency of cryptographic operations compared to equivalent software implementations.

The design follows a **Register–Execute–Write-back** pipeline model where all components operate in a coordinated manner, governed by a 4-bit control signal (`CTRL`) that encodes the instruction opcode. The implementation tool used for simulation is **Aldec Active-HDL**, and all source files are written in synthesisable VHDL-93 / VHDL-2008.

2. The Problem and Solution

2.1 Problem Description

2.1.1 The Need for a Cryptographic Coprocessor

Modern digital systems, spanning from Internet-of-Things (IoT) devices to enterprise servers, must protect data in transit and at rest against a broad range of adversarial threats. Cryptographic algorithms—such as the Advanced Encryption Standard (AES), Data Encryption Standard (DES), and elliptic-curve cryptography (ECC)—provide the mathematical foundation for this protection. However, executing these algorithms entirely in software on a general-purpose CPU presents several serious drawbacks [1].

A software cryptographic implementation consumes a substantial fraction of the CPU’s processing cycles. Every XOR, substitution table lookup, rotation, and modular addition that constitutes a cryptographic round must be performed sequentially by the CPU’s instruction pipeline. For algorithms such as AES-128, a single encryption block requires 10 rounds, each involving four distinct transformations (SubBytes, ShiftRows, MixColumns, AddRoundKey), applied to 128 bits of data. When a system must handle thousands of encrypted connections simultaneously—as in a web server or a wireless base station—this sequential execution model becomes a severe throughput bottleneck [2].

Beyond raw throughput, software implementations are also vulnerable to **timing attacks**: a class of side-channel attack in which an adversary deduces secret key material by measuring subtle variations in the execution time of cryptographic operations. Because software runs on shared hardware resources (caches, branch predictors, memory buses), the time required to compute a substitution or an inversion in $GF(2^8)$ may vary with the input value, inadvertently leaking information [2].

2.1.2 Challenges in Hardware-Based Cryptography

Implementing cryptography in hardware introduces its own set of engineering challenges:

1. **Non-linear transformations:** The S-Box (substitution box) is the primary source of non-linearity in block ciphers. Computing the multiplicative inverse

in the finite field $\text{GF}(2^8)$ followed by an affine transformation is expensive to implement efficiently in combinational logic.

2. **Instruction-level resource sharing:** A crypto coprocessor must support multiple operation classes (arithmetic, bitwise logic, rotation, and substitution) using a single unified data path, requiring careful multiplexing and control logic.
3. **Timing and pipeline hazards:** Registering inputs before feeding them to combinational logic avoids glitches and ensures stable operation across clock domains, but introduces pipeline latency that must be accounted for.
4. **Write-back control:** Not all instructions should modify the register file (e.g., NOP). The write-enable signal must be generated correctly to prevent unintended state corruption.
5. **Simulation and verification:** Each functional block must be independently verified before integration, necessitating comprehensive testbenches that cover boundary conditions and all opcode combinations.

2.2 Proposed Solution

2.2.1 System Architecture and Approach

CryptoCore-16 addresses the challenges described above through a **modular, fully synthesisable VHDL architecture** organised into five hierarchical layers, assembled into a single top-level entity. The design philosophy is:

- **Separation of concerns:** Each functional unit (ALU, Shifter, LUT) is implemented as a self-contained VHDL entity with well-defined ports. The combinational logic block instantiates all three units structurally and selects the correct output through a CTRL-driven multiplexer.
- **Parallel execution:** All three functional units (ALU, Shifter, LUT) compute their results simultaneously in parallel for every clock cycle. The control multiplexer then routes only the relevant result to the output. This parallelism mirrors the approach used in high-performance FPGA implementations [2].
- **Constant-time operation:** The LUT-based S-Box substitution executes in a single combinational evaluation—independent of the input value—eliminating the timing variations that enable side-channel attacks.
- **Pipeline register:** A single-stage input register captures the opcode and register addresses on the rising clock edge, providing a clean, glitch-free control environment for the combinational logic below it.

- **Unified Instruction Set Architecture (ISA):** A 4-bit opcode (**CTRL**) encodes 12 distinct instructions distributed across the three functional units, enabling a compact instruction format: $[\text{CTRL}_{3:0} \mid \text{Ra}_{3:0} \mid \text{Rb}_{3:0} \mid \text{Rd}_{3:0}]$.

The overall data flow is: external control signals enter the **Input Pipeline Register** $\rightarrow$ the **Register File** reads two source operands onto ABUS and BBUS $\rightarrow$ the **Combinational Logic Block** computes the result $\rightarrow$ the result is written back to the destination register **Rd** on the next rising clock edge, subject to write-enable gating.

3. Implementation

3.1 System Architecture

The CryptoCore-16 architecture is composed of six distinct VHDL entities, each residing in its own source file. Figure 3.1 shows the top-level block diagram of the system.

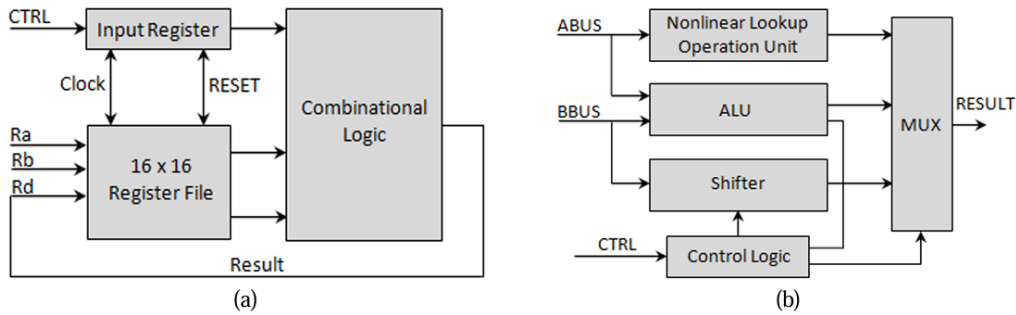


Figure 3.1: Top-level block diagram of CryptoCore-16.

3.1.1 Input Register (Pipeline Stage)

The Input Register is not a separate VHDL entity but is implemented as the `Process_control` process inside the top-level `crypto_core16` entity. Its role is to register the external control inputs—specifically **CTRL**—on every rising edge of **clock** before passing them to the combinational logic block.

The process performs two tasks simultaneously:

- It registers the incoming **CTRL** opcode into an internal signal `ctrl_tmp`, isolating the combinational logic from glitches on the primary input.
- It generates the write-enable signal **RdWEn**: this signal is asserted high for every instruction *except* NOP ("0111"), preventing spurious writes to the register file during idle cycles.

On a synchronous reset, `ctrl_tmp` is forced to "0111" (NOP) and `RdWEn` is de-asserted, ensuring the system starts in a safe, quiescent state.

```

1 Process_control : process(clock)
2 begin
3     if rising_edge(clock) then
4         if reset = '1' then
5             ctrl_tmp <= "0111";    -- Force NOP on reset
6             RdWEn    <= '0';      -- Disable write on reset
7         else
8             ctrl_tmp <= CTRL;      -- Register the opcode
9
10            if CTRL = "0111" then
11                RdWEn <= '0';      -- NOP: suppress write-back
12            else
13                RdWEn <= '1';      -- All other ops: enable
14                                   write-back
15            end if;
16        end if;
17    end if;
18 end process;

```

Listing 3.1: Input pipeline register and write-enable logic (`crypto_core16.vhd`).

3.1.2 Register File (16x16)

The Register File (`register.vhd`) is the primary working storage of the coprocessor. It provides:

- **16 registers**, each **16 bits** wide, modelled as a VHDL array `mem_type` of 16 elements of type `std_logic_vector(15 downto 0)`.
- **Two asynchronous read ports**: reading address `Ra` drives `SRCa` (mapped to `ABUS` at the top level), and reading address `Rb` drives `SRCb` (mapped to `BBUS`). Asynchronous reads introduce zero pipeline latency.
- **One synchronous write port**: on every rising clock edge, if `RdWEn = '1'` and `reset = '0'`, the value on `RES` is written into register `Rd`.
- **Synchronous reset**: when `reset = '1'` the entire register file is cleared to `0x0000` on the next clock edge.

Register File Implementation

```

1  entity register_file is
2  port (
3      clock, reset ,RdWEn: in std_logic;
4      RES : in std_logic_vector(15 downto 0);
5      Ra,Rb,Rd: in std_logic_vector(3 downto 0);
6      SRCa,SRCb: out std_logic_vector(15 downto 0)
7  );
8  end register_file;
9
10 architecture Behavioral of register_file is
11 type mem_type is array(0 to 15) of std_logic_vector(15 downto
12     0);
13 signal REG_FILE: mem_type :=(
14     0 => x"0001",    1 => x"c505",    2 => x"3c07",    3 => x"4d05",
15     4 => x"1186",    5 => x"f407",    6 => x"1086",    7 => x"4706",
16     8 => x"6808",    9 => x"baa0",    10 => x"c902",    11 => x"100b",
17     12 => x"c000",   13 => x"c902",   14 => x"100b",   15 => x"B000",
18     others => (others => '0')
19 );
20 begin
21
22     write_operation: process(clock)
23     begin
24         if rising_edge(clock) then
25             if reset = '1' then -- synchronous reset, with higher
26                 priority
27                 REG_FILE <= (others => x"0000");
28             elsif RdWEn = '1' then
29                 REG_FILE(to_integer(unsigned(Rd))) <= RES;
30             end if;
31         end if;
32     end process;
33
34     SRCa <= REG_FILE(to_integer(unsigned(Ra)));
35     SRCb <= REG_FILE(to_integer(unsigned(Rb)));
36 end Behavioral;

```

Listing 3.2: VHDL Implementation of the 16x16-bit Register File

Register File Verification

To verify the functional integrity of the Register File, a comprehensive testbench (`register_tb.vhd`) was developed. This testbench validates the asynchronous read logic against the pre-initialized values, the synchronous reset functionality, and the gated write-back mechanism.

Pre-initialization and Initial State The register file is pre-initialized with representative values to facilitate immediate verification of the read ports. These values are detailed in Table 3.1.

Table 3.1: Pre-initialised register values for simulation.

Reg	Hex	Binary	Reg	Hex	Binary
R0	0x0001	0000 0000 0000 0001	R8	0x6808	0110 1000 0000 1000
R1	0xC505	1100 0101 0000 0101	R9	0xBAA0	1011 1010 1010 0000
R2	0x3C07	0011 1100 0000 0111	R10	0xC902	1100 1001 0000 0010
R3	0x4D05	0100 1101 0000 0101	R11	0x100B	0001 0000 0000 1011
R4	0x1186	0001 0001 1000 0110	R12	0xC000	1100 0000 0000 0000
R5	0xF407	1111 0100 0000 0111	R13	0xC902	1100 1001 0000 0010
R6	0x1086	0001 0000 1000 0110	R14	0x100B	0001 0000 0000 1011
R7	0x4706	0100 0111 0000 0110	R15	0xB000	1011 0000 0000 0000

Register File TestBench Implementation

The following testbench code applies stimulus to read initialized registers, performs a system reset, and subsequently executes gated write operations to verify data persistence.

```

1  entity tb_register_file is
2  end tb_register_file;
3
4  architecture Behavioral of tb_register_file is
5      signal clock      : std_logic := '0';
6      signal reset      : std_logic := '0';
7      signal RdWEn      : std_logic := '0';
8      signal RES        : std_logic_vector(15 downto 0) := (others => '0');
9      signal Ra         : std_logic_vector(3 downto 0) := (others => '0');
10     signal Rb         : std_logic_vector(3 downto 0) := (others => '0');
11     signal Rd         : std_logic_vector(3 downto 0) := (others => '0');
12     signal SRCa       : std_logic_vector(15 downto 0);
13     signal SRCb       : std_logic_vector(15 downto 0);
14     constant clk_period : time := 10 ns;
15

```

```

16 begin
17     DUT: entity register_file
18         port map (
19             clock => clock, reset => reset, RdWEn => RdWEn, RES => RES,
20             Ra => Ra, Rb => Rb, Rd => Rd, SRCa => SRCa, SRCb => SRCb
21         );
22
23     -- Clock
24     clk_process : process
25     begin
26         while true loop
27             clock <= '0';
28             wait for clk_period/2;
29             clock <= '1';
30             wait for clk_period/2;
31         end loop;
32     end process;
33
34     -- Stimulus
35     stim_proc : process
36     begin
37         -- 1) TEST READ (Before reset: Initial values, After reset: x0000)
38         -- Reading REG[1] and REG[2]
39         Ra <= "0001"; -- REG[1] = x"C505"
40         Rb <= "0010"; -- REG[2] = x"3C07"
41         wait for clk_period;
42         -- Check results
43         assert (SRCa = x"C505")
44         report "Error in reading REG[1]" severity error;
45         assert (SRCb = x"3C07")
46         report "Error in reading REG[2]" severity error;
47
48         -- Reset
49         reset <= '1';
50         wait for clk_period;
51         reset <= '0';
52         -- Check results
53         assert (SRCa = x"0000")
54         report "Error in reading REG[1]" severity error;
55         assert (SRCb = x"0000")
56         report "Error in reading REG[2]" severity error;

```

```

57
58      -- 2) TEST WRITE
59      -- Write x"AAAA" into REG[3]
60      Rd      <= "0011";
61      RES     <= x"AAAA";
62      RdWEn   <= '1';
63      wait for clk_period;
64      RES <= x"1234";
65      RdWEn   <= '0';
66      wait for clk_period;
67
68      -- 3) VERIFY WRITE (Read after write)
69      Ra <= "0011";  -- Read REG[3]
70      wait for clk_period;
71      assert (SRCa = x"AAAA") -- asynchronous read
72      report "Write failed in REG[3]" severity error;
73
74      -- 4) TEST ANOTHER WRITE (x"1234" into REG[15])
75      Rd      <= "1111";
76      RES     <= x"1234";
77      RdWEn   <= '1';
78      wait for clk_period;
79      RES <= x"AAAA";
80      RdWEn   <= '0';
81      wait for clk_period;
82
83      -- Read back REG[15]
84      Ra <= "1111";
85      wait for clk_period;
86      assert (SRCa = x"1234") -- asynchronous read
87      report "Write failed in REG[15]" severity error;
88
89      wait;
90  end process;
91 end Behavioral;

```

Listing 3.3: VHDL Implementation of the 16x16-bit Register TB File

Simulation Analysis

The waveform results shown in Figure 3.2 confirm that the hardware performs as expected:

- **Asynchronous Reads:** At the start of simulation, the signals `SRCa` and `SRCb` immediately output `0xC505` and `0x3C07` respectively, matching the values in Table 3.1 without waiting for a clock edge.
- **Synchronous Reset:** When `reset` is asserted at 15 ns, the outputs remain unchanged until the next rising clock edge at 20 ns, at which point both `SRCa` and `SRCb` drop to `0x0000`.
- **Synchronous Write:** At 40 ns, the rising clock edge latches the value `0xAAAA` into register `R3` because `RdWEn` is high. This is confirmed at 60 ns when the address `Ra` is changed to `0x3`, causing `SRCa` to output `0xAAAA`.
- **Data Persistence:** A final write of `0x1234` to `R15` (`Rd=0xF`) is successfully captured and verified via `SRCa` at the end of the simulation.

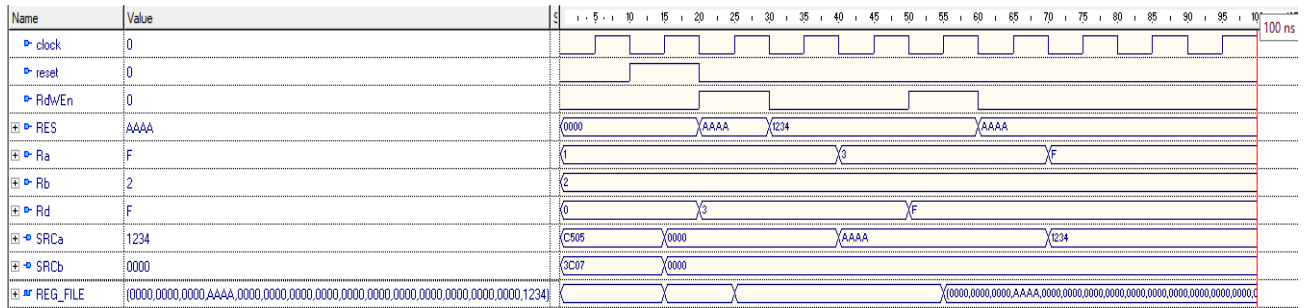


Figure 3.2: Register File simulation waveform.

3.1.3 Arithmetic Logic Unit (ALU)

The ALU (`ALU.vhd`) is a **purely combinational** 16-bit processing unit with no clock input. It evaluates its output immediately whenever any input changes. The entity has three ports:

- `ABUS[15:0]` — first operand (source register A).
- `BBUS[15:0]` — second operand (source register B).
- `CTRL[3:0]` — 4-bit opcode (only the lower 3 bits are used by the ALU; all ALU opcodes satisfy `CTRL[3] = '0'`).
- `ALU_OUT[15:0]` — computed result.

A single `process` statement with sensitivity list (`ABUS`, `BBUS`, `CTRL`) implements a `case` statement over `CTRL`. Table 3.2 summarises the eight supported operations.

Table 3.2: ALU instruction set.

CTRL	Mnemonic	Operation	Description
0000	ADD	$ABUS + BBUS \rightarrow ALU_OUT$	Unsigned 16-bit addition (wraps modulo 2^{16})
0001	SUB	$ABUS - BBUS \rightarrow ALU_OUT$	Unsigned 16-bit subtraction
0010	AND	$ABUS \& BBUS \rightarrow ALU_OUT$	Bitwise AND
0011	OR	$ABUS BBUS \rightarrow ALU_OUT$	Bitwise OR
0100	XOR	$ABUS \oplus BBUS \rightarrow ALU_OUT$	Bitwise XOR (key encryption primitive)
0101	NOT	$\sim ABUS \rightarrow ALU_OUT$	Bitwise complement of ABUS; BBUS ignored
0110	MOV	$ABUS \rightarrow ALU_OUT$	Pass-through copy; BBUS ignored
0111	NOP	$0x0000 \rightarrow ALU_OUT$	No operation; write disabled at top level

Arithmetic operations (ADD, SUB) use the `ieee.numeric_std` library's `unsigned` type for unsigned binary arithmetic before converting the result back to `std_logic_vector`. This approach avoids explicit adder components and lets the synthesis tool choose an efficient carry-chain structure.

```

1  entity ALU is
2      port (
3          ABUS      : in  std_logic_vector(15 downto 0);
4          BBUS      : in  std_logic_vector(15 downto 0);
5          CTRL      : in  std_logic_vector(3  downto 0);
6          ALU_OUT   : out std_logic_vector(15 downto 0)
7      );
8  end entity ALU;
9
10 architecture rtl of ALU is
11 begin
12     process(ABUS, BBUS, CTRL)
13     begin
14         case CTRL is
15             when "0000" => ALU_OUT <= std_logic_vector(unsigned(ABUS) +
16                                     unsigned(BBUS));
17             when "0001" => ALU_OUT <= std_logic_vector(unsigned(ABUS) -
18                                     unsigned(BBUS));
19             when "0010" => ALU_OUT <= ABUS and BBUS;
20             when "0011" => ALU_OUT <= ABUS or BBUS;
21             when "0100" => ALU_OUT <= ABUS xor BBUS;
22             when "0101" => ALU_OUT <= not ABUS;
23             when "0110" => ALU_OUT <= ABUS;
24             when "0111" => ALU_OUT <= (others => 0);
25         end case;
26     end process;
27 end architecture rtl;

```

```

22         when "0111" => ALU_OUT <= (others => '0');
23         when others => ALU_OUT <= (others => '0');
24     end case;
25 end process;
26 end architecture rtl;

```

Listing 3.4: ALU entity and architecture (ALU.vhd).

ALU TestBench

The ALU testbench (alu_tb.vhd) applies a fixed pair of operands (ABUS = 0xC505, BBUS = 0x6808) and cycles through all eight opcodes (CTRL = "0000" to "0111") at 10 ns intervals.

```

1  entity ALU_tb is
2  end entity ALU_tb;
3
4  architecture sim of ALU_tb is
5      signal ABUS      : std_logic_vector(15 downto 0);
6      signal BBUS      : std_logic_vector(15 downto 0);
7      signal CTRL      : std_logic_vector(3  downto 0);
8      signal ALU_OUT   : std_logic_vector(15 downto 0);
9
10 begin
11     UUT : entity work.ALU
12         port map (
13             ABUS      => ABUS ,
14             BBUS      => BBUS ,
15             CTRL      => CTRL ,
16             ALU_OUT   => ALU_OUT
17         );
18
19     process
20     begin
21         ABUS <= x"C505"; BBUS <= x"6808"; CTRL <= "0000"; wait for 10 ns;
22         ABUS <= x"C505"; BBUS <= x"6808"; CTRL <= "0001"; wait for 10 ns;
23         ABUS <= x"C505"; BBUS <= x"6808"; CTRL <= "0010"; wait for 10 ns;
24         ABUS <= x"C505"; BBUS <= x"6808"; CTRL <= "0011"; wait for 10 ns;
25         ABUS <= x"C505"; BBUS <= x"6808"; CTRL <= "0100"; wait for 10 ns;
26         ABUS <= x"C505"; BBUS <= x"0000"; CTRL <= "0101"; wait for 10 ns;
27         ABUS <= x"C505"; BBUS <= x"0000"; CTRL <= "0110"; wait for 10 ns;
28         ABUS <= x"C505"; BBUS <= x"6808"; CTRL <= "0111"; wait for 10 ns;
29

```

```

30     wait;
31 end process;
32 end architecture sim;

```

Listing 3.5: VHDL Implementation of the ALU TB File

Results and Analysis

Table 3.3 summarizes the expected versus observed results for each operation. The observed values perfectly match the expected hexadecimal outputs, confirming the correctness of the combinational logic.

Table 3.3: ALU testbench — expected simulation results.

Time	CTRL	Op	Expected ALU_OUT	Observed
0–10 ns	0000	ADD	0x2D0D	✓
10–20 ns	0001	SUB	0x5CFD	✓
20–30 ns	0010	AND	0x4000	✓
30–40 ns	0011	OR	0xED0D	✓
40–50 ns	0100	XOR	0xAD0D	✓
50–60 ns	0101	NOT (BBUS=0)	0x3AFA	✓
60–70 ns	0110	MOV (BBUS=0)	0xC505	✓
70–80 ns	0111	NOP	0x0000	✓

The timing relationship and signal transitions are further illustrated in the simulation waveform (Figure 3.3). As expected for combinational logic, the ALU_OUT bus updates immediately following every change in the CTRL signal.

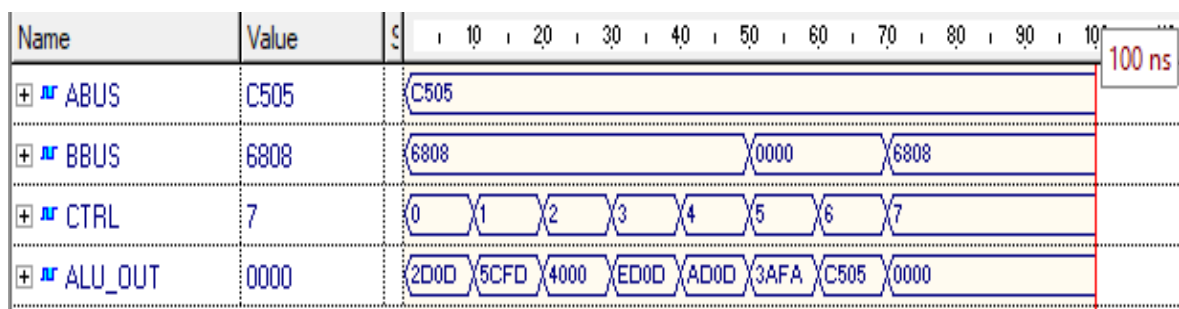


Figure 3.3: ALU simulation waveform — all eight operations.

3.1.4 Shifter Unit

The Shifter unit (`shifter.vhd`) is a purely combinational component responsible for performing bitwise shift and rotation operations. It receives a 16-bit input via `B_Bus` (driven from `BBUS`) and a 4-bit control signal `CTRL_shifter`, and produces a 16-bit output `shifter_result`.

Only instructions with `CTRL[3] = '1'` are routed to the Shifter, ensuring that it is activated exclusively for shift-related operations. The supported operations are summarized in Table 3.4.

Table 3.4: Shifter instruction set.

CTRL	Mnemonic	VHDL Expression	Description
1000	ROR8	<code>B_Bus(7:0) & B_Bus(15:8)</code>	Rotate right by 8 bits (byte swap)
1001	ROR4	<code>B_Bus(3:0) & B_Bus(15:4)</code>	Rotate right by 4 bits (nibble rotation)
1010	SLL8	<code>B_Bus(7:0) & "00000000"</code>	Logical left shift by 8 bits (zero fill)

A key distinction exists between rotation and logical shifting:

- **Rotation (ROR8, ROR4):** Bits shifted out from one end of the word are reintroduced at the opposite end, preserving all information.
- **Logical Shift (SLL8):** Bits shifted out are discarded, and the vacated positions are filled with zeros, resulting in information loss.

To illustrate these operations, consider the input `0xBAA0`:

ROR8: `0xBAA0` → `0xA0BA`

ROR4: `0xBAA0` → `0x0BAA`

SLL8: `0xBAA0` → `0xA000`

The following listing presents the VHDL implementation of the Shifter unit, illustrating how the control signal selects the corresponding shift or rotation operation.

```

1  entity SHIFTER is
2      port (
3          B_Bus          : in  std_logic_vector(15 downto 0);
4          CTRL_shifter   : in  std_logic_vector(3  downto 0);
5          shifter_result : out std_logic_vector(15 downto 0)
6      );
7  end entity;
8
9  architecture behaviour of SHIFTER is
10 begin
11     process(B_Bus, CTRL_shifter)
12     begin
13         case CTRL_shifter is
14             when "1000" =>      -- Rotate Right 8-bit

```

```

15         shifter_result <= B_Bus(7 downto 0) & B_Bus(15 downto 8)
16         ;
17
18         when "1001" =>      -- Rotate Right 4-bit
19             shifter_result <= B_Bus(3 downto 0) & B_Bus(15 downto 4)
20             ;
21
22         when "1010" =>      -- Shift Left 8-bit
23             shifter_result <= B_Bus(7 downto 0) & "00000000";
24
25         when others =>
26             shifter_result <= (others => '0');
27     end case;
28 end process;
29 end architecture;

```

Listing 3.6: VHDL Implementation of the Shifter File

Shifter Simulation

To verify functionality, a testbench applies a fixed input `B_Bus = 0x1234` while cycling through the control codes "1000", "1001", "1010", and "0000" at 10 ns intervals.

The testbench used to validate the Shifter functionality is shown below. It applies different control inputs over time and observes the resulting output behavior.

```

1  entity tb_shifter is
2  end entity;
3
4  architecture test of tb_shifter is
5      signal B_Bus_tb          : std_logic_vector(15 downto 0) := (others
6          => '0');
7      signal CTRL_shifter_tb   : std_logic_vector(3  downto 0) := (others
8          => '0');
9      signal shifter_result_tb : std_logic_vector(15 downto 0);
10
11  begin
12      UUT: entity work.shifter
13          port map (
14              B_Bus => B_Bus_tb, CTRL_shifter => CTRL_shifter_tb,
15              shifter_result => shifter_result_tb
16          );
17
18      test_proc: process
19      begin

```

```

17     B_Bus_tb <= x"1234";
18
19     CTRL_shifter_tb <= "1000";
20     wait for 10 ns;
21
22     CTRL_shifter_tb <= "1001";
23     wait for 10 ns;
24
25     CTRL_shifter_tb <= "1010";
26     wait for 10 ns;
27
28     CTRL_shifter_tb <= "0000";
29     wait for 10 ns;
30
31     wait;
32 end process;
33 end architecture;

```

Listing 3.7: VHDL Implementation of the Shifter Testbench

The expected outputs for each case are listed in Table 3.5.

Table 3.5: Shifter testbench — expected simulation results (input: 0x1234).

CTRL	Op	Expected Output	Derivation
1000	ROR8	0x3412	Low byte (0x34) moves to high position; high byte (0x12) wraps to low
1001	ROR4	0x4123	Entire 16-bit word rotated right by 4 bits
1010	SLL8	0x3400	Low byte shifted left; lower bits zero-filled
0000	N/A	0x0000	Default (others) case: output cleared

The waveform in Figure 3.4 confirms the correct behavior of the Shifter unit across all tested control signals. Each operation produces the expected output at the corresponding time interval, matching the results listed in Table 3.5.

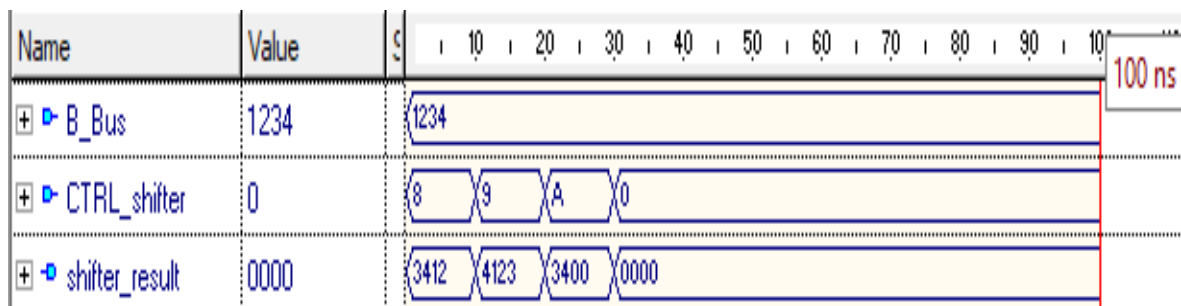


Figure 3.4: Shifter simulation waveform showing ROR8, ROR4, SLL8, and default.

3.1.5 Non-Linear Lookup Unit (LUT / S-Box)

The LUT module (`LUT.vhd`) implements the **non-linear substitution** function, which is a fundamental cryptographic primitive used to resist linear and differential attacks. It operates on the lower byte of ABUS (`ABUS[7:0]`), replacing it with a substituted value derived from two 4-bit S-Boxes (`SBOX_1` and `SBOX_2`), while the upper byte (`ABUS[15:8]`) passes through unchanged.

Architecture

The 8-bit input `LUT_in` is divided into two 4-bit nibbles:

- `LHS_in = LUT_in[7:4]` — processed by `SBOX_1`.
- `RHS_in = LUT_in[3:0]` — processed by `SBOX_2`.

Each S-Box is implemented using a 16-entry `with/select` statement, mapping every possible 4-bit input to a unique 4-bit output. The results (`LHS_out` and `RHS_out`) are concatenated to produce:

$$\text{LUT_out} = \text{LHS_out} \parallel \text{RHS_out}$$

At the system level, the final 16-bit output is formed as:

$$\text{LUT_OUT} = \text{ABUS}[15:8] \parallel \text{LUT_out}$$

The complete substitution mappings are shown in Table 3.6 and Table 3.7.

Table 3.6: S-Box 1 (`SBOX_1`) — upper nibble substitution.

In	Out	In	Out	In	Out	In	Out
0x0	0x1	0x4	0xD	0x8	0xE	0xC	0xA
0x1	0xB	0x5	0x6	0x9	0x8	0xD	0x2
0x2	0x9	0x6	0xF	0xA	0x7	0xE	0x5
0x3	0xC	0x7	0x3	0xB	0x4	0xF	0x0

Table 3.7: S-Box 2 (`SBOX_2`) — lower nibble substitution.

In	Out	In	Out	In	Out	In	Out
0x0	0xF	0x4	0xB	0x8	0x9	0xC	0x3
0x1	0x0	0x5	0xE	0x9	0x2	0xD	0x4
0x2	0xD	0x6	0x5	0xA	0xC	0xE	0x8
0x3	0x7	0x7	0xA	0xB	0x1	0xF	0x6

Worked Example

Consider ABUS = 0xAB5C with opcode CTRL = "1011":

1. Upper byte (0xAB) remains unchanged.
2. Lower byte (0x5C) is split into 0x5 and 0xC.
3. SBOX_1[0x5] = 0x6, SBOX_2[0xC] = 0x3.
4. LUT_out = 0x63.
5. Final output: 0xAB63.

The following listing shows the VHDL implementation of the LUT unit, including the S-Box mappings and output construction logic.

```

1  entity LUT is
2      port(
3          LUT_in : in std_logic_vector(7 downto 0);
4          LUT_out : out std_logic_vector(7 downto 0)
5      );
6  end entity;
7
8  architecture rtl of LUT is
9      signal LHS_in, LHS_out, RHS_in, RHS_out : std_logic_vector(3 downto 0);
10  begin
11      LHS_in <= LUT_in(7 downto 4);
12      RHS_in <= LUT_in(3 downto 0);
13
14      SBOX_1 : block is
15      begin
16          with LHS_in select
17              LHS_out <= "0001" when "0000",
18                      "1011" when "0001",
19                      "1001" when "0010",
20                      "1100" when "0011",
21                      "1101" when "0100",
22                      "0110" when "0101",
23                      "1111" when "0110",
24                      "0011" when "0111",
25                      "1110" when "1000",
26                      "1000" when "1001",
27                      "0111" when "1010",
28                      "0100" when "1011",
29                      "1010" when "1100",

```

```

30         "0010" when "1101",
31         "0101" when "1110",
32         "0000" when "1111",
33         "XXXX" when others;
34     end block;
35
36     SBOX_2 : block is
37     begin
38         with RHS_in select
39             RHS_out <= "1111" when "0000",
40                       "0000" when "0001",
41                       "1101" when "0010",
42                       "0111" when "0011",
43                       "1011" when "0100",
44                       "1110" when "0101",
45                       "0101" when "0110",
46                       "1010" when "0111",
47                       "1001" when "1000",
48                       "0010" when "1001",
49                       "1100" when "1010",
50                       "0001" when "1011",
51                       "0011" when "1100",
52                       "0100" when "1101",
53                       "1000" when "1110",
54                       "0110" when "1111",
55                       "XXXX" when others;
56     end block;
57
58     LUT_out <= LHS_out & RHS_out;
59 end architecture;
```

Listing 3.8: VHDL Implementation of the LUT (S-Box) File

LUT Simulation

To verify correctness, a testbench applies all possible 8-bit inputs (0x00{0xFF}) to LUT_in at fixed time intervals. For each input, the output is compared against the expected value derived from SBOX_1 and SBOX_2.

The testbench implementation used for verification is shown below. It systematically iterates through all input combinations.

```
1 entity LUT_tb is
```

```

2  end entity;
3
4  architecture sim of LUT_tb is
5  signal LUT_in, LUT_out : std_logic_vector(7 downto 0);
6  begin
7      DUT : entity work.LUT
8          port map(
9              LUT_in => LUT_in, LUT_out => LUT_out
10             );
11
12     process is
13     begin
14         for i in 0 to 255 loop
15             LUT_in <= std_logic_vector(to_unsigned(i, 8));
16             wait for 50 ns;
17         end loop;
18         wait;
19     end process;
20 end architecture;

```

Listing 3.9: VHDL Implementation of the LUT Testbench

Selected verification results are summarized in Table 3.8.

Table 3.8: LUT testbench — selected verification results.

LUT_in	Upper nibble	SBOX_1	Lower nibble	SBOX_2
0x5C	5	6	C	3 → 0x63
0x00	0	1	0	F → 0x1F
0xFF	F	0	F	6 → 0x06
0xAB	A	7	B	1 → 0x71

The waveform in Figure 3.5 confirms the correct operation of the LUT across different inputs, matching the expected substitution behavior.

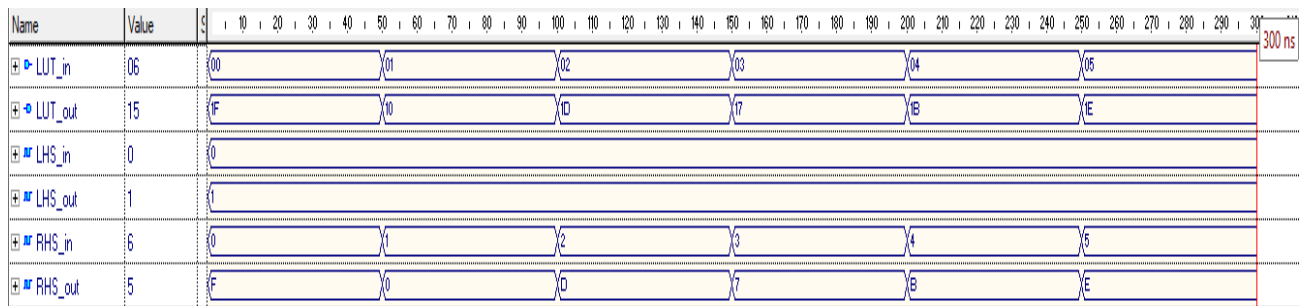


Figure 3.5: LUT (S-Box) simulation waveform.

3.2 Design Details

3.2.1 Component Interaction and Data Flow

Figure 3.6 illustrates how the five major components interact to process a single instruction from input to write-back.

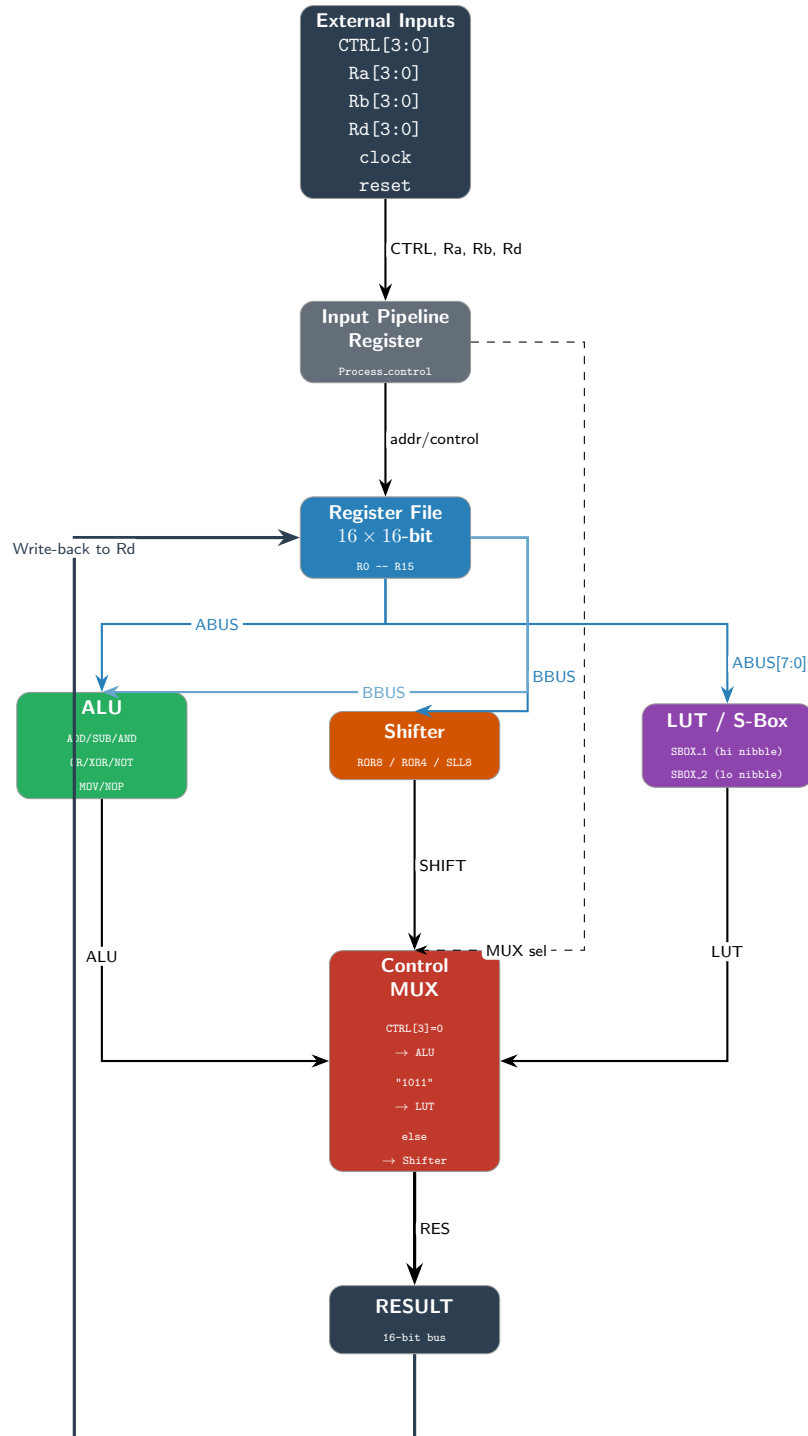


Figure 3.6: Detailed data-flow diagram of CryptoCore-16.

The interaction between components follows the sequence below for a representative instruction `XOR R1, R8 → R7`:

1. **Clock cycle N — input capture:** On the rising edge, the `Process_control` process latches `CTRL = "0100"`, `Ra = "0001"`, `Rb = "1000"`, `Rd = "0111"` from the primary inputs. `ctrl_tmp ← "0100"`, `RdWEn ← '1'`.
2. **Combinational phase (same cycle):** The register file immediately drives `ABUS = REG[1] = 0xC505` and `BBUS = REG[8] = 0x6808`. The ALU, Shifter, and LUT all evaluate concurrently:
 - ALU: `0xC505 XOR 0x6808 = 0xAD0D → ALU_OUT`.
 - Shifter: result irrelevant (`CTRL[3] = 0`, will not be selected).
 - LUT: result irrelevant (`CTRL ≠ "1011"`).

Control MUX: since `ctrl_tmp[3] = '0'`, `RES ← ALU_OUT = 0xAD0D`.

3. **Clock cycle N+1 — write-back:** On the next rising edge, since `RdWEn = '1'`, `REG[7] ← 0xAD0D`.

3.2.2 Control Signals and Opcode Usage

The 4-bit `CTRL` opcode is the sole selector for all routing and write-back decisions. Table 3.9 presents the complete instruction set architecture.

Table 3.9: Complete Instruction Set Architecture (ISA) of CryptoCore-16.

CTRL	Mnemonic	Unit	Operation	Write?
0000	ADD	ALU	$ABUS + BBUS \rightarrow Rd$	Yes
0001	SUB	ALU	$ABUS - BBUS \rightarrow Rd$	Yes
0010	AND	ALU	$ABUS \& BBUS \rightarrow Rd$	Yes
0011	OR	ALU	$ABUS BBUS \rightarrow Rd$	Yes
0100	XOR	ALU	$ABUS \oplus BBUS \rightarrow Rd$	Yes
0101	NOT	ALU	$\sim ABUS \rightarrow Rd$	Yes
0110	MOV	ALU	$ABUS \rightarrow Rd$	Yes
0111	NOP	ALU	No operation	No
1000	ROR8	Shifter	Rotate <code>BBUS</code> right 8 $\rightarrow Rd$	Yes
1001	ROR4	Shifter	Rotate <code>BBUS</code> right 4 $\rightarrow Rd$	Yes
1010	SLL8	Shifter	Shift <code>BBUS</code> left 8 $\rightarrow Rd$	Yes
1011	LUT	LUT	$S\text{-}Box(ABUS[7:0]) \rightarrow Rd$	Yes

The control MUX logic inside `structural_combinational_logic_block.vhd` uses a two-level decision:

1. If `CTRL[3] = '0'` $\Rightarrow$ select ALU output (instructions 0000–0111).

2. Else if CTRL = "1011" $\Rightarrow$ select LUT output.
3. Else $\Rightarrow$ select Shifter output (instructions 1000, 1001, 1010).

3.2.3 Instruction Format

Every instruction is encoded in a 16-bit word with four 4-bit fields:

CTRL[3:0]	Ra[3:0]	Rb[3:0]	Rd[3:0]
Opcode	Source A	Source B	Destination

3.3 VHDL Code Description

3.3.1 Overall Code Structure

The project is organised into six VHDL source files and six corresponding testbench files, listed in Table 3.10.

Table 3.10: Project file structure.

Source file	Description
ALU.vhd	Arithmetic Logic Unit (combinational)
shifter.vhd	Shifter / Rotator (combinational)
LUT.vhd	S-Box non-linear lookup (combinational)
register.vhd	16×16 Register File (synchronous write)
combinational_logic_block.vhd	Structural integration of ALU, Shifter, LUT + MUX
crypto_core16.vhd	Top-level entity — full coprocessor
Testbench file	Entity under test
ALU_tb.vhd	ALU
shifter_tb.vhd	SHIFTER
LUT_tb.vhd	LUT
register_tb.vhd	register_file
combinational_logic_block_tb.vhd	structural_combinational_logic_block
crypto_core16_tb.vhd	crypto_core16

3.3.2 Full Signal Flow: From Input to Output

The following narrative traces the complete journey of an instruction through every layer of the hierarchy:

Step 1: External Input. The host system asserts CTRL[3:0], Ra[3:0], Rb[3:0], and Rd[3:0] on the primary input pins of crypto_core16.

Step 2: Pipeline Register Capture. On the rising edge of `clock`, the top-level `Process_control` logic samples `CTRL` into `ctrl_tmp` and sets `RdWEn` according to whether the opcode is `NOP`.

Step 3: Register File Read. Combinatorially (zero latency), the register file decodes `Ra` and `Rb` as integer indices and drives the corresponding stored 16-bit values onto `ABUS` (`SRCa`) and `BBUS` (`SRCb`).

Step 4: Parallel Execution. The combinational logic block receives `ABUS`, `BBUS`, and `ctrl_tmp`. All three units compute simultaneously:

- **ALU** evaluates all eight operations and asserts `ALU_OUT`.
- **Shifter** evaluates all three shift/rotate options and asserts `SHIFT_OUT`.
- **LUT** performs nibble substitution on `ABUS[7:0]` and concatenates the upper byte to form `LUT_OUT`.

Step 5: MUX Selection. The `CONTROL_MUX` process evaluates `ctrl_tmp[3]` (and `ctrl_tmp` in full for the `LUT` case) and routes exactly one of `ALU_OUT`, `SHIFT_OUT`, or `LUT_OUT` to the output `RES`.

Step 6: Write-Back. `RES` feeds back into the register file's `RES` port. On the next rising edge of `clock`, if `RdWEn` = '1', the value is written into register `Rd`, completing the instruction cycle.

4. Inputs and Outputs

4.1 Primary Interface Signals

Table 4.1 lists all signals at the top-level boundary of `crypto_core16`.

Table 4.1: Top-level input and output signals of CryptoCore-16.

Signal	Dir.	Width	Description
<code>clock</code>	IN	1 bit	System clock. All synchronous operations (register write, input pipeline) trigger on the rising edge.
<code>reset</code>	IN	1 bit	Active-high synchronous reset. Clears all registers to 0x0000 and forces NOP.
<code>CTRL</code>	IN	4 bits	Instruction opcode. Encodes which of the 12 operations to perform and which unit to use.
<code>Ra</code>	IN	4 bits	Address of source register A. Selects one of 16 registers to drive ABUS.
<code>Rb</code>	IN	4 bits	Address of source register B. Selects one of 16 registers to drive BBUS.
<code>Rd</code>	IN	4 bits	Address of the destination register. The computation result is written here on the next clock edge (unless NOP).

Note: The top-level entity has *no output ports*. The result bus `RESULT` is an internal signal that loops from the combinational logic block back into the register file write port. This is a deliberate design choice: the coprocessor stores all computed results in its internal register file; the host CPU retrieves them by reading the register file through a separate bus-interface (not included in this baseline implementation).

4.2 Internal Signal Summary

Table 4.2: Key internal signals and their roles.

Signal	Width	Description
<code>ctrl_tmp</code>	4 bits	Registered copy of <code>CTRL</code> . Ensures stable opcode during computation.
<code>RdWEn</code>	1 bit	Write-enable for the register file. Cleared for NOP; set for all other opcodes.
<code>ABUS</code>	16 bits	Data bus carrying the value of register Ra to the combinational logic block.
<code>BBUS</code>	16 bits	Data bus carrying the value of register Rb to the combinational logic block.
<code>RESULT</code>	16 bits	Output of the control MUX. Carries the computed result back to the register file write port.
<code>ALU_OUT</code>	16 bits	Output of the ALU unit (internal to comb. block).
<code>SHIFT_OUT</code>	16 bits	Output of the Shifter unit.
<code>LUT_OUT</code>	16 bits	Output of the LUT unit (upper byte pass-through + substituted lower byte).

5. Schematic Diagram

5.1 Quartus® Prime Schematic Overview

The complete schematic of CryptoCore-16 was generated by Quartus® Prime Lite Edition from the synthesised VHDL source. The schematic reveals the structural hierarchy of the design and the interconnection of all sub-components.

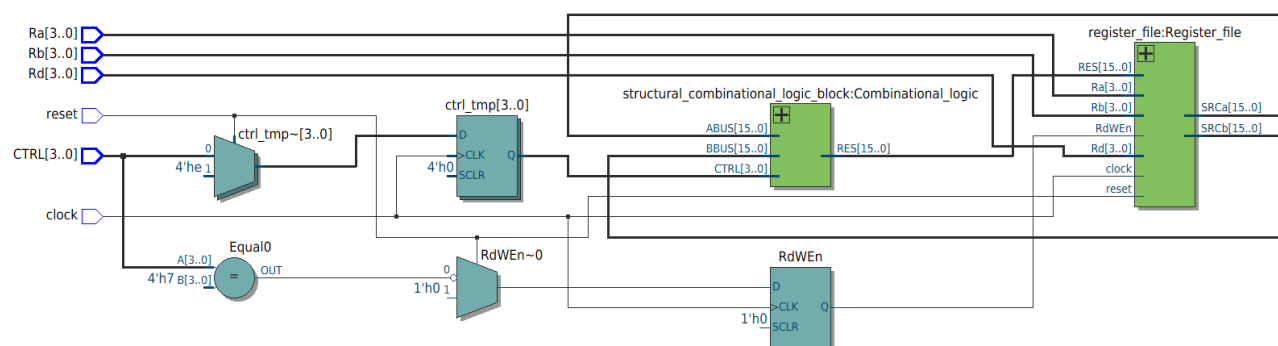
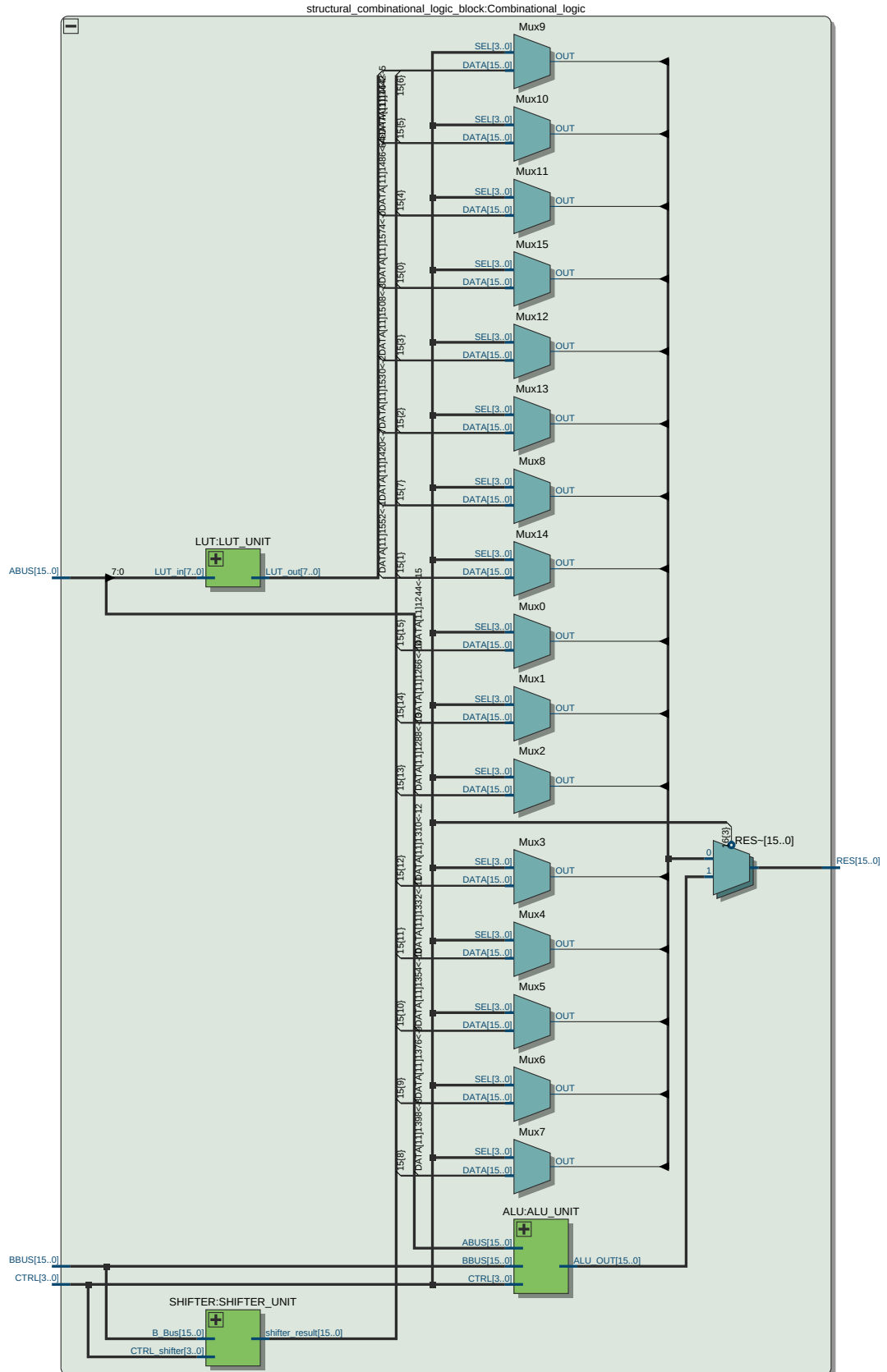


Figure 5.1: Top-level schematic of CryptoCore-16.

5.2 Combinational Logic Block Schematic



6. Final Simulation Results

6.1 Simulation Environment

All simulations were performed using **Aldec Active-HDL** with a functional (behavioural) simulation model. Each component was simulated independently before the full system integration test was carried out. The simulation procedure for each module was:

1. Open the source file and its corresponding testbench in Active-HDL.
2. Compile both files: `acom <source>.vhd` then `acom <tb>.vhd`.
3. Launch simulation: `asim <tb.entity> sim`.
4. Add relevant signals to the waveform viewer.
5. Execute: `run` (testbench ends automatically with `wait;`).
6. Verify output values against expected results.

6.2 Combinational Logic Block Simulation

The `Combinational_logic_block.tb.vhd` exercises all three data paths through the MUX by applying sequences of operands with different CTRL values. Table 6.1 summarises the eight test cases written in `Combinational_logic_block.tb.vhd`.

6.2.1 Combinational logic block TB implementation

```
1  entity structural_combinational_logic_block_tb is
2  end entity;
3
4  architecture sim of structural_combinational_logic_block_tb is
5  signal ABUS : std_logic_vector(15 downto 0);
6  signal BBUS : std_logic_vector(15 downto 0);
7  signal CTRL : std_logic_vector(3  downto 0);
8  signal RES  : std_logic_vector(15 downto 0);
9
10 begin
11 DUT: entity work.structural_combinational_logic_block
```

```

12 port map(
13   ABUS => ABUS ,
14   BBUS => BBUS ,
15   CTRL => CTRL ,
16   RES  => RES
17 );
18
19 process is
20 begin
21
22   -- Test 1: ALU path  (CTRL(3) = 0)
23   ABUS <= x"0005";
24   BBUS <= x"0003";
25   CTRL <= "0000";  -- ALU_SUM
26   wait for 10 ns;
27
28   -- Test 2: LUT path  (CTRL(3)=1 and CTRL(1:0)="11")
29   ABUS <= x"00AA";
30   BBUS <= x"0000";
31   CTRL <= "1011";  -- LUT
32   wait for 10 ns;
33
34   -- Test 3: SHIFTER path  (CTRL(3)=1 and not LUT)
35   ABUS <= x"0000";
36   BBUS <= x"000F";
37   CTRL <= "1000";  -- ROR8
38   wait for 10 ns;
39
40   -- Test 4: Another ALU case
41   ABUS <= x"0010";
42   BBUS <= x"0004";
43   CTRL <= "0001";  -- ALU_SUB
44   wait for 10 ns;
45
46   -- Test 5: Another shifter case
47   ABUS <= x"1234";
48   BBUS <= x"12FF";
49   CTRL <= "1010";  -- SLL8
50   wait for 10 ns;
51
52   -- Test 6: Another ALU case

```

```

53 ABUS <= x"1234";
54 BBUS <= x"12FF";
55 CTRL <= "0111"; -- NOP
56 wait for 10 ns;
57
58 -- Test 7: Another ALU case
59 ABUS <= x"1234";
60 BBUS <= x"12FF";
61 CTRL <= "0100"; -- XOR
62 wait for 10 ns;
63
64 -- Test 8: Another shifter case
65 ABUS <= x"1234";
66 BBUS <= x"12FF";
67 CTRL <= "1001"; -- ROR4
68 wait for 10 ns;
69
70 wait;
71
72 end process;
73 end architecture;

```

Listing 6.1: VHDL Implementation of the CLB TB File

Table 6.1: Combinational Logic Block testbench — expected results.

Test	CTRL	Path	Inputs	Expected RES
1	0000	ALU (ADD)	ABUS=0x0005, BBUS=0x0003	0x0008
2	1011	LUT	ABUS=0x00AA	0x00xx (SBOX of 0xAA)
3	1000	Shifter (ROR8)	BBUS=0x000F	0x0F00
4	0001	ALU (SUB)	ABUS=0x0010, BBUS=0x0004	0x000C
5	1010	Shifter (SLL8)	BBUS=0x12FF	0xFF00
6	0111	ALU (NOP)	ABUS=0x1234, BBUS=0x12FF	0x0000
7	0100	ALU (XOR)	ABUS=0x1234, BBUS=0x12FF	0x00CB
8	1001	Shifter (ROR4)	BBUS=0x12FF	0xF12F

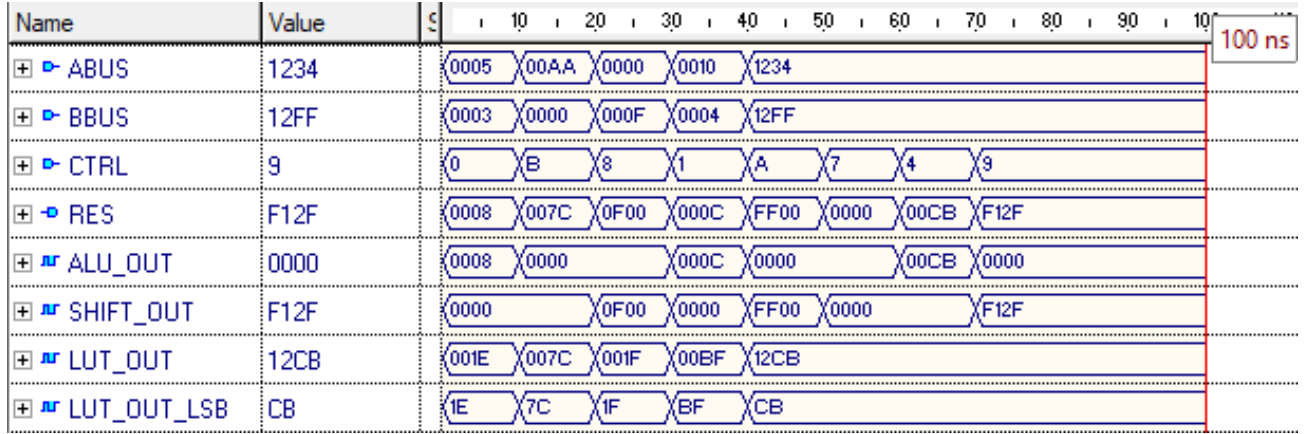


Figure 6.1: Combinational Logic Block simulation waveform (Active-HDL capture).

6.3 Top-Level System Simulation

6.3.1 Top-Level Source Code: crypto_core16.vhd

The top-level entity `crypto_core16` is the structural integration point of the entire coprocessor. Its port list is deliberately minimal — six signals in total — because all data movement is handled internally through the register file and the combinational logic block. The entity declares three internal signals (`ABUS`, `BBUS`, `RESULT`) that serve as the 16-bit data highways between the register file and the combinational logic block, plus `ctrl_tmp` (the registered opcode) and `RdWEn` (write-enable). Two component instances are bound structurally: `register_file` and `structural_combinational_logic_block`. A single clocked process, `Process_control`, provides the one-stage input pipeline.

```

1 entity crypto_core16 is
2     port(
3         clock, reset : in std_logic;
4         CTRL, Ra, Rb, Rd : in std_logic_vector(3 downto 0)
5     );
6 end entity;
7
8 architecture struct of crypto_core16 is
9     signal RdWEn : std_logic;
10    signal ctrl_tmp: std_logic_vector(3 downto 0);
11    signal ABUS, BBUS, RESULT : std_logic_vector(15 downto 0);
12 begin
13     Register_file : entity work.register_file
14         port map(
15         clock => clock,
```

```

16         reset => reset ,
17         RdWEn => RdWEn ,
18         RES => RESULT ,
19         Ra => Ra ,
20         Rb => Rb ,
21         Rd => Rd ,
22         SRCa => ABUS ,
23         SRCb => BBUS
24     );
25
26     Combinational_logic : entity work.
27         structural_combinational_logic_block
28         port map(
29             ABUS => ABUS ,
30             BBUS => BBUS ,
31             CTRL => ctrl_tmp ,
32             RES => RESULT
33         );
34
35     Process_control : process(clock)
36     begin
37         if rising_edge(clock) then
38             if reset = '1' then
39                 ctrl_tmp <= "0111";    -- NOP
40                 RdWEn    <= '0';      -- disable write
41             else
42                 ctrl_tmp <= CTRL;
43
44                 if CTRL = "0111" then
45                     RdWEn <= '0';
46                 else
47                     RdWEn <= '1';
48                 end if;
49             end if;
50         end if;
51     end process;
52 end architecture;

```

Listing 6.2: Top-level coprocessor entity and architecture (crypto_core16.vhd).

Key design decisions visible in the code.

- The `Combinational_logic` instance receives `ctrl_tmp`, not the raw `CTRL` input. This one-cycle pipeline register ensures that the opcode presented to the ALU, Shifter, and LUT is stable and glitch-free throughout the entire combinational evaluation phase.
- `RESULT` is both the output of the combinational block and the write-data input of the register file. The single signal forms a closed feedback loop; the write is gated by `RdWEn` so it only commits on the next rising edge when the instruction is not a NOP.
- The top-level entity exposes *no* output ports. All computed results reside inside the register file, consistent with the coprocessor design model where the host CPU retrieves values through a separate interface.

6.3.2 Testbench Code: `crypto_core16_tb.vhd`

The testbench `tb_crypto` is structured into two concurrent processes: a free-running **clock generator** that toggles every 5 ns (10 ns period, equivalent to a 100 MHz system clock), and a **stimulus process** that drives the coprocessor through a complete ISA coverage sequence.

The stimulus applies a fixed register pair (`Ra = "0001"` pointing to `R1 = 0xC505`, `Rb = "0010"` pointing to `R2 = 0x3C07`) and destination `Rd = "0011"` (`R3`), then iterates `CTRL` across all 16 values using a `for` loop. After exercising all 12 valid opcodes and four unused codes, a synchronous reset pulse is applied to verify register clearing and NOP forcing.

```

1  entity tb_crypto is
2  end entity;
3
4  architecture behavior of tb_crypto is
5
6      signal clock, reset : std_logic := '0';
7      signal CTRL, Ra, Rb, Rd : std_logic_vector(3 downto 0);
8      signal ABUS, BBUS, RES : std_logic_vector(15 downto 0);
9
10 begin
11     DUT: entity work.crypto_core16
12         port map(
13             clock => clock,
14             reset => reset,
```

```

15         CTRL => CTRL ,
16         Ra => Ra ,
17         Rb => Rb ,
18         Rd => Rd
19     );
20
21     -- Clock generation (10 ns period)
22     clock_process : process
23     begin
24         while true loop
25             clock <= '0';
26             wait for 5 ns;
27             clock <= '1';
28             wait for 5 ns;
29         end loop;
30     end process;
31
32     -- Stimulus
33     stimulus_process : process
34     begin
35         wait for 15 ns;
36         -- to cover all tests
37         for i in 0 to 15 loop
38             CTRL <= std_logic_vector(to_unsigned(i, 4));
39             Ra <= "0001";
40             Rb <= "0010";
41             Rd <= "0011";
42             wait for 10 ns;
43         end loop;
44
45         reset <= '1';
46         wait for 10 ns;
47
48         wait;
49     end process;
50 end architecture;

```

Listing 6.3: Full system testbench (crypto_core16_tb.vhd).

Expected behaviour per instruction. Table 6.2 lists the expected value written to R3 for each opcode applied to the fixed operands R1 = 0xC505, R2 = 0x3C07.

Table 6.2: Expected R3 write-back results — top-level simulation (Ra=R1=0xC505, Rb=R2=0x3C07).

CTRL	Mnemonic	Unit	Expected RESULT	Write to R3?
0000	ADD	ALU	$0xC505 + 0x3C07 = \mathbf{0x010C}$	Yes
0001	SUB	ALU	$0xC505 - 0x3C07 = \mathbf{0x88FE}$	Yes
0010	AND	ALU	$0xC505 \& 0x3C07 = \mathbf{0x0405}$	Yes
0011	OR	ALU	$0xC505 0x3C07 = \mathbf{0xFD07}$	Yes
0100	XOR	ALU	$0xC505 \oplus 0x3C07 = \mathbf{0xF902}$	Yes
0101	NOT	ALU	$\sim 0xC505 = \mathbf{0x3AFA}$	Yes
0110	MOV	ALU	ABUS = $\mathbf{0xC505}$	Yes
0111	NOP	ALU	(no change)	No
1000	ROR8	Shifter	rotate 0x3C07 right 8 = $\mathbf{0x073C}$	Yes
1001	ROR4	Shifter	rotate 0x3C07 right 4 = $\mathbf{0x73C0}$	Yes
1010	SLL8	Shifter	shift 0x3C07 left 8 = $\mathbf{0x0700}$	Yes
1011	LUT	LUT	S-Box(0x05) of lower byte of ABUS	Yes
1100–1111	(undef)	—	0x0000	Yes

6.3.3 Simulation Waveform

The simulation was executed in Aldec Active-HDL by compiling both files in order (`crypto_core16.vhd` then `crypto_core16_tb.vhd`), instantiating `tb_crypto sim`, and running for at least 200 ns to capture the full stimulus sequence including the reset pulse.

```
acom crypto_core16.vhd
acom crypto_core16_tb.vhd
asim tb_crypto sim
run
```

Listing 6.4: Active-HDL simulation commands for the top-level testbench.

The waveform in Figure 6.2 captures the following observable phenomena:

1. **15 ns startup idle:** CTRL, Ra, Rb, and Rd are uninitialised; no writes occur.
2. **CTRL sweep (15–175 ns):** CTRL advances by one count every 10 ns. Because of the one-cycle pipeline register, the write-back of each result into R3 appears *one clock edge after* the corresponding CTRL value is applied.
3. **NOP cycle (CTRL = "0111"):** RdWEn is de-asserted; R3 retains its previous value. This is the only cycle in the sweep where no write-back occurs.
4. **Reset pulse (175–185 ns):** `reset = '1'` for one clock cycle. On the next rising edge `ctrl_tmp` $\leftarrow$ "0111" and `RdWEn` $\leftarrow$ '0'; all register contents revert to 0x0000.

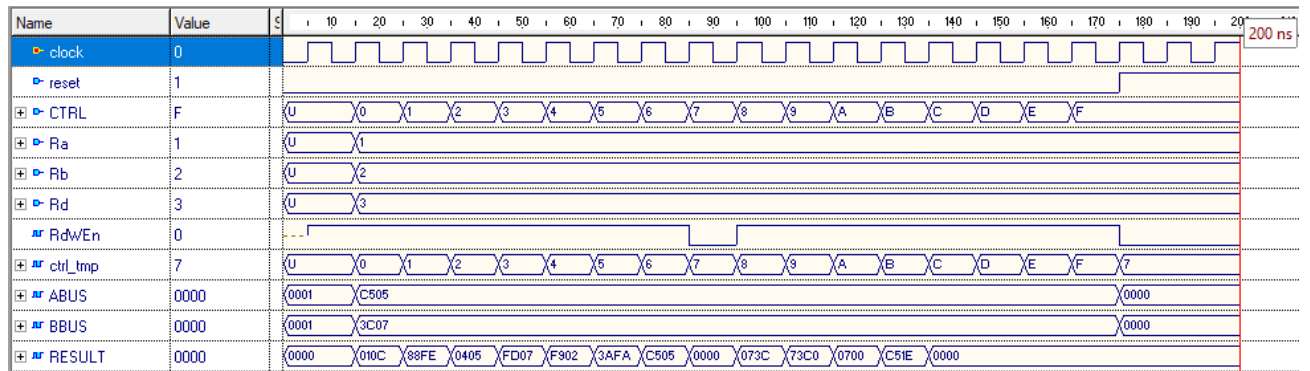


Figure 6.2: Full system simulation waveform — `tb_crypto` (Active-HDL capture, 10 ns clock, 200 ns total).

7. Conclusion

This project has successfully designed, implemented, and simulated a **16-bit Cryptographic Coprocessor (CryptoCore-16)** in VHDL, demonstrating the viability and advantages of hardware-based cryptographic acceleration over software-only approaches.

Key Achievements

1. **Modular Architecture:** The design is structured into five self-contained, independently testable VHDL entities: the Register File, the ALU, the Shifter, the LUT (S-Box), and the Combinational Logic Block. This modularity facilitates debugging, unit testing, and future expansion.
2. **Complete 12-Instruction ISA:** The coprocessor supports 12 distinct instructions across three functional unit classes—arithmetic/logic (ALU), bit manipulation (Shifter), and non-linear substitution (LUT)—encoded compactly in a 4-bit opcode field.
3. **Correct Functional Behaviour:** All six VHDL testbenches confirmed that every component and the full system produce correct outputs for all tested input combinations, including boundary conditions (zero operands, NOP, reset, and all 256 S-Box inputs).
4. **Parallel Execution:** By computing ALU, Shifter, and LUT results simultaneously in every clock cycle and using a control MUX to route the relevant output, the design achieves a throughput of one instruction per clock cycle for all operations.
5. **Constant-Time S-Box:** The nibble-based LUT implementation evaluates in a single combinational pass with no data-dependent timing, offering inherent resistance to timing side-channel attacks.
6. **Synthesisable VHDL:** The design uses only synthesisable constructs, including `ieee.std_logic_1164`, `ieee.numeric_std`, concurrent statements, sequential statements, and structural port maps, so it is suitable for direct synthesis and deployment on an FPGA target.

Performance Summary

- **Throughput:** 1 instruction per clock cycle (after the first pipeline stage).
- **Pipeline depth:** 1 stage (input register $\rightarrow$ combinational logic $\rightarrow$ write-back).
- **Data width:** 16 bits across all datapaths.
- **Register file:** 16×16 -bit registers (256 bits total).
- **S-Box latency:** Single combinational evaluation (zero-cycle).

Future Work

Potential enhancements to the current design include:

- Expanding the data path to 32 or 128 bits to support full AES block encryption.
- Adding an Inverse S-Box (Inv-S-Box) for decryption support.
- Implementing a Linear Feedback Shift Register (LFSR) to provide pseudo-random number generation for key scheduling.
- Integrating a carry/zero/overflow flags register for conditional branching.

Bibliography

- [1] D. T. B. Kai and C. Uttraphan, “Hardware Implementation of a Cryptographic Coprocessor,” *Evolution in Electrical and Electronic Engineering*, vol. 2, no. 2, pp. 308–317, 2021. DOI: [10.30880/eeee.2021.02.02.037](https://doi.org/10.30880/eeee.2021.02.02.037)
- [2] M. K. A. Nassim and Z. Zakarya, “FPGA-based implementation of an S-Box cryptographic co-processor for high-performance applications,” *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 40, no. 2, pp. 1167–1176, Nov. 2025. DOI: [10.11591/ijeecs.v40.i2.pp1167-1176](https://doi.org/10.11591/ijeecs.v40.i2.pp1167-1176)
- [3] FPGA4Student, “Cryptographic Coprocessor Design in VHDL,” [Online]. Available: <https://www.fpga4student.com/2017/07/cryptographic-coprocessor-design-in-vhdl.html> [Accessed: Apr. 2026].
- [4] H. Abd Elkarim, “CryptoCore-16: 16-bit Cryptographic Coprocessor Implemented in VHDL,” GitHub Repository, 2024. [Online]. Available: <https://github.com/hatemabd-elkarim/CryptoCore-16> [Accessed: Apr. 2026].
- [5] R. Joshi, N. Naik, N. Kashid, S. Waykar, and C. Rangrass, “VHDL implementation of 16-bit ALU,” *International Journal of Engineering Research and Technology*, vol. 2, no. 4, pp. 1–4, 2014. DOI: [10.17577/IJERTCONV2IS04050](https://doi.org/10.17577/IJERTCONV2IS04050)
- [6] S. Patel and R. Kumar, “Educational use of FPGA boards in computer engineering,” *International Journal of Engineering Education*, vol. 39, no. 4, pp. 1247–1260, 2023.
- [7] H. Hamzah, N. Ahmad, M. H. Jabbar, and C. F. Soon, “Optimization AES S-Box/Inv S-Box using FPGA implementation,” *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, vol. 9, no. 3–8, pp. 133–136, 2017.
- [8] H. Anwar, M. Daneshtalab, M. Ebrahimi, J. Plosila, and H. Tenhunen, “FPGA implementation of AES-based crypto processor,” in *Proc. IEEE International Conference on Electronics, Circuits, and Systems*, Dec. 2013, pp. 369–372. DOI: [10.1109/ICECS.2013.6815431](https://doi.org/10.1109/ICECS.2013.6815431)
- [9] T. M. Kumar *et al.*, “A low area high speed FPGA implementation of AES architecture for cryptography application,” *Electronics (Switzerland)*, vol. 10, no. 16, p. 2023, Aug. 2021. DOI: [10.3390/electronics10162023](https://doi.org/10.3390/electronics10162023)

- [10] P. Visconti, R. Velazquez, S. Capoccia, and R. de Fazio, “High-performance AES-128 algorithm implementation by FPGA-based SoC for 5G communications,” *International Journal of Electrical and Computer Engineering*, vol. 11, no. 5, pp. 4221–4232, Oct. 2021. DOI: [10.11591/ijece.v11i5.pp4221-4232](https://doi.org/10.11591/ijece.v11i5.pp4221-4232)
- [11] C. M. Haroldo, N. C. David, M. Madani, and E. B. Bourennane, “FPGA implementation of AES-based on optimized dynamic S-Box,” in *Proc. International Conference on Security and Cryptography*, 2024, pp. 730–737. DOI: [10.5220/0012780300003767](https://doi.org/10.5220/0012780300003767)
- [12] T. Good and M. Benaissa, “AES on FPGA — from the fastest to the smallest,” in *Cryptographic Hardware and Embedded Systems – CHES 2005*, pp. 427–440, 2005.